

Trabajo Práctico N.º 5

Operaciones de alta (inserción) en MongoDB

Materia	Bases de Datos Avanzadas
Tema	insertOne, insertMany, ObjectId, write concern y validación de esquema
Modalidad de trabajo	Individual o en parejas
Herramientas	MongoDB Community / MongoDB Atlas, mongosh, MongoDB Compass

1. Introducción

Los tres trabajos prácticos anteriores dejaron sentadas las bases: cómo se diseña un documento (TP N.º 2), cómo se opera desde el shell (TP N.º 3) y cómo se opera desde una interfaz gráfica (TP N.º 4). Este trabajo práctico profundiza específicamente en la operación de alta (create/insert), que hasta ahora se utilizó de forma superficial, cubriendo aspectos que todo desarrollador o administrador de MongoDB debe dominar: el comportamiento del identificador `_id`, la diferencia entre inserciones ordenadas y no ordenadas, el manejo de errores de duplicados, el nivel de confirmación de escritura (write concern) y la validación de documentos mediante un validator basado en JSON Schema (retomando el TP N.º 2).

2. Objetivos de aprendizaje

- Comprender el rol del campo `_id` y del tipo `ObjectId`.
- Diferenciar `insertOne()`, `insertMany()` e `insertMany()` con `ordered: false`.
- Interpretar y resolver errores de clave duplicada (E11000).
- Comprender el concepto de write concern y su impacto en la durabilidad de una escritura.
- Crear una colección con un validator de JSON Schema y observar el rechazo de documentos inválidos.
- Realizar una carga masiva de datos desde un archivo externo.

3. Marco teórico

3.1 El campo `_id` y el tipo `ObjectId`

Todo documento de MongoDB posee un campo `_id` que actúa como clave primaria de la colección y es único dentro de ella. Si no se especifica al insertar, MongoDB genera automáticamente un valor de tipo `ObjectId`: una estructura de 12 bytes que codifica, entre otros datos, un timestamp de creación, por lo que los `ObjectId` son, en la práctica, ordenables cronológicamente.

```
db.productos.insertOne({ nombre: "Mouse", precio: 12000 })
// El documento resultante incluye, por ejemplo:
// { _id: ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"), nombre: "Mouse", precio: 12000 }
```

También es válido asignar manualmente un `_id` propio (string, number, etc.), siempre que se garantice su unicidad dentro de la colección; esto es habitual cuando el dominio ya cuenta con un identificador natural, como un código de producto o un legajo.

3.2 insertOne() vs. insertMany()

Método	Comportamiento	Retorno
<code>insertOne(doc)</code>	Inserta un único documento.	<code>{ acknowledged, insertedId }</code>
<code>insertMany([docs])</code>	Inserta varios documentos en una sola operación.	<code>{ acknowledged, insertedIds }</code>

3.3 Inserciones ordenadas vs. no ordenadas

Por defecto, `insertMany()` es ordenada (`ordered: true`): si un documento falla (por ejemplo, por `_id` duplicado), la operación se detiene y los documentos posteriores del array no se insertan. Con `ordered: false`, MongoDB intenta insertar todos los documentos posibles, reportando al final la lista de los que fallaron, sin detenerse ante el primer error.

```
db.productos.insertMany([
  { _id: "P-01", nombre: "Teclado" },
  { _id: "P-01", nombre: "Teclado duplicado" }, // provocará error E11000
  { _id: "P-02", nombre: "Mouse" }
], { ordered: false })
// Con ordered:false, P-01 (primero) y P-02 sí se insertan; solo falla el duplicado.
// Con ordered:true (default), P-02 NO se llega a insertar.
```

3.4 Write concern

El write concern determina cuántos nodos de un clúster deben confirmar una escritura para que el cliente la considere exitosa. En un entorno de aprendizaje con una única instancia, este concepto es menos visible, pero es central en producción, donde MongoDB normalmente se despliega como un replica set.

Valor	Significado
w: 1 (default)	Basta con que el nodo primario confirme la escritura en memoria.
w: 0	No se espera confirmación alguna ("fire and forget"); máxima velocidad, mínima garantía.
w: "majority"	Se espera confirmación de la mayoría de los nodos del replica set antes de responder al cliente; máxima durabilidad.
j: true	Exige además que la escritura quede asentada en el journal de disco del nodo, no solo en memoria.

```
db.productos.insertOne(
  { nombre: "Monitor 24\"", precio: 145000 },
  { writeConcern: { w: "majority", j: true } }
)
```

3.5 Validación de esquema al crear la colección

En el TP N.º 1 se diseñaron esquemas de validación en JSON Schema de forma puramente conceptual. MongoDB permite aplicar ese mismo esquema de manera efectiva: al crear una colección con `db.createCollection()`, se puede indicar la opción `validator`, de modo que cualquier inserción (o actualización) que no cumpla el esquema sea rechazada por el servidor.

```
db.createCollection("productos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "precio"],
```

```
    properties: {
      nombre: { bsonType: "string", minLength: 2 },
      precio: { bsonType: ["int", "double"], minimum: 0 },
      stock: { bsonType: "int", minimum: 0 }
    }
  },
  validationLevel: "strict",
  validationAction: "error"
})
```

Nótese la diferencia de vocabulario respecto al JSON Schema estándar del TP N.º 1: MongoDB envuelve el esquema en la clave `$jsonSchema` y utiliza `bsonType` en lugar de `type`, ya que valida contra los tipos de BSON (por ejemplo, distinguiendo `int` de `double`), no contra los tipos genéricos de JSON.

validationAction

Con `validationAction: "warn"`, MongoDB permite igualmente la escritura pero registra una advertencia en el log; con `"error"` (el valor por defecto), la rechaza directamente. Esto resulta útil para migrar gradualmente datos existentes hacia un esquema más estricto.

4. Consignas de trabajo

Ejercicio 1 — `_id` automático vs. `_id` manual

Inserte un documento en una colección de prueba sin especificar `_id` y otro especificando un `_id` propio (por ejemplo, un código de producto). Consulte ambos con `find()` y explique la diferencia observada en el campo `_id` de cada uno.

Ejercicio 2 — Error de clave duplicada

Intente insertar dos documentos con el mismo `_id` mediante dos llamadas a `insertOne()`. Capture el mensaje de error E11000 completo y explique, en sus palabras, qué información brinda ese mensaje.

Ejercicio 3 — Ordered vs. unordered

Prepare un array de 5 documentos donde el tercero tenga un `_id` duplicado respecto de otro ya existente en la colección. Ejecute `insertMany()` dos veces sobre colecciones equivalentes: una vez con la configuración por defecto (`ordered`) y otra con `ordered: false`. Compare y documente cuántos documentos quedaron insertados en cada caso.

Ejercicio 4 — Colección con validación de esquema

Cree una colección `productos_validados` aplicando un validator de JSON Schema que exija: nombre (string, obligatorio), precio (numérico, mínimo 0, obligatorio) y categoría (uno de un conjunto cerrado de valores mediante enum).

Intente insertar: (a) un documento válido, que debe aceptarse; (b) un documento sin el campo `precio`, que debe rechazarse; (c) un documento con categoría fuera del enum permitido, que también debe rechazarse. Capture los tres resultados.

Ejercicio 5 — Write concern

Investigue y documente (no es necesario un clúster replicado real) qué ocurriría con una escritura configurada con `w: "majority"` en un replica set de 3 nodos si uno de los nodos secundarios está caído. Redacte una respuesta de al menos 5 líneas.

Ejercicio 6 — Carga masiva desde archivo

Genere (o utilice el de un ejercicio anterior) un archivo de al menos 20 documentos de la entidad de su elección en formato JSON, y cárguelo en una colección mediante `mongoimport`, o bien mediante un script que lea el archivo y ejecute `insertMany()`.

Verifique la cantidad final de documentos con `countDocuments()` y documente el comando utilizado.

```
$ mongoimport --db badavanzada --collection carga_masiva --file datos.json --jsonArray
```

5. Forma de entrega

- Archivo o script con los comandos ejecutados en cada ejercicio.
- Archivo de datos utilizado en el Ejercicio 6.
- Documento (PDF o Word) con las capturas de pantalla y las explicaciones solicitadas en los Ejercicios 1, 2, 3 y 5.
- Plazo de entrega y vía de entrega: a definir por la cátedra en el aula virtual.

6. Criterios de evaluación

Criterio	Descripción	Puntos
Manejo de _id	Comprende y demuestra correctamente la diferencia entre ObjectId automático y _id manual.	15
Errores de duplicados y ordered/unordered	Identifica y explica correctamente el comportamiento ante errores de escritura.	20
Validación de esquema	El validator rechaza y acepta los documentos según lo esperado; buen uso de \$jsonSchema.	30
Write concern	Explicación conceptual correcta y coherente del impacto en durabilidad y disponibilidad.	15
Carga masiva	Carga exitosa y verificable de un conjunto de datos externo.	20
Total		100